UNIVERSIDAD NACIONAL AUTÓNOMA DE MÉXICO

FACULTAD DE CIENCIAS

Práctica 6
Flex, Bison, AST y análisis semántico.

ALUMNO

Gallegos Cortes José Antonio
Angel Maldonado Gerardo De Jesus

PROFESOR

Víctor Germán Mijangos de la Cruz.

AYUDANTES

Teresa Becerril Torres
Adrián Martínez Manzo

ASIGNATURA

Compiladores.

1. Descripción general de la solución

En esta práctica implementamos un analizador para un lenguaje pequeño escrito completamente en código Morse. La idea fue usar una secuencias de puntos, rayas y diagonales como representación de los tokens del lenguaje, de momento solo asignamos algunas letras a los elementos sintácticos básicos (tipos, palabras clave, operadores), y definimos un formato específico para los identificadores y literales numéricos. El ejecutable final se llama `validator`. Lee un programa desde la entrada estándar y se comporta de la siguiente manera:

- si la entrada es correcta, imprime una representación legible del AST
- si encuentra un error sintáctico o semántico, reporta un mensaje claro con línea y columna.

Idea del lenguaje

El lenguaje que diseñamos es imperativo y deliberadamente pequeño, queremos que sea legible al menos saber donde hay separaciones. Incluye:

- tipos `int` y `float`
- declaraciones de variables
- asignaciones
- expresiones aritméticas con `+`, `-`, `*` y `/`
- bloques con llaves
- estructuras `if` y `while`.

Lo importante es que todo eso se escribe en Morse. Por ejemplo, en la implementación:

- `.-` representa `int`
- `..-.` representa `float`
- `..` representa `if`
- `.-` representa `while`
- `-...` representa `=`
- `...-` representa `;`.

Además, para evitar ambigüedades, los identificadores y literales numéricos se codifican con prefijos especiales. Por ejemplo, los identificadores inician con `..-..`, los enteros con `-.-.-` y los flotantes con `.-..-.`. Esto permite que el lexer los reconozca sin confusión, aunque hace que el código Morse sea un poco más largo, pero aún no sabemos si lo vamos a mantener así o si vamos a usar un formato más sencillo.

Archivos principales

La práctica quedó así:

- `lexer.l`: define los tokens en Flex y lleva el rastreo de línea y columna
- `parser.y`: define la gramática en Bison, construye el AST y realiza parte del chequeo semántico
- `ast.h`: contiene la estructura del AST, los tipos semánticos y la tabla de símbolos por ámbitos
- `Makefile`: compila el ejecutable `validator`, ojo, para este es necesario tener instalado flex y bison.
- `examples/`: contiene entradas de prueba correctas e incorrectas, los nombres de los archivos dicen el tipo de error que mandan o si son correctas.

Funcionamiento general

El flujo completo del programa es el siguiente:

1. Flex lee el código Morse y lo transforma en tokens.
2. Bison consume esos tokens y verifica que respeten la gramática.
3. Durante cada reducción importante se crea un nodo del AST.
4. En paralelo se consulta y actualiza una tabla de símbolos por ámbitos para validar declaraciones, usos de variables y compatibilidad de tipos.
5. Al final, si no hubo errores, se imprime el AST. Si hubo errores, se imprime el error correspondiente.

2. Fase léxica y fase sintáctica

Fase léxica con Flex

En `lexer.l` definímos reglas para reconocer secuencias de Morse y convertirlas en tokens del lenguaje. La estrategia fue separar claramente tres grupos:

- palabras reservadas y símbolos simples, por ejemplo `.-`, `..-.`, `-...` o `...-`
- identificadores codificados en Morse
- literales enteros y flotantes codificados en Morse.

Para identificadores y números usamos una función auxiliar que parte la secuencia por diagonales y luego intenta decodificar cada fragmento con una tabla Morse. Por ejemplo:

- `..-../-..-` se traduce al identificador `x`
- `-.-.-/.--` se traduce al entero `1`
- `.-..-./.--/.-.-.-/..--` se traduce al flotante `1.2`.

También activamos el manejo de ubicaciones con `yylloc`. Cada token actualiza línea y columna, de modo que si aparece un error el parser puede reportarlo con precisión.

Fase sintáctica con Bison

En `parser.y` definimos una gramática pequeña para la práctica. La estructura general del lenguaje es:

- una lista de sentencias
- sentencias de declaración, asignación, bloque, `if` y `while`
- expresiones aritméticas y agrupación con paréntesis.

La gramática base se puede resumir así:

```
program     -> stmt_list
stmt        -> declaration
            | assignment
            | block
            | if ( expr ) stmt
            | while ( expr ) stmt

block       -> { stmt_list }
declaration -> type id
            | type id = expr
assignment  -> id = expr
expr        -> expr + expr
```

```
            | expr - expr
            | expr * expr
            | expr / expr
            | id | int_lit | float_lit | ( expr )
```

Para evitar problemas con ambigüedad aritmética declaramos precedencia para `+`, `-`, `*` y `/`. Con eso Bison construye correctamente expresiones como `x + 1 * 2` sin necesidad de reglas redundantes.

Construcción del AST durante el parseo

Cada reducción importante en el archivo `.y` devuelve un puntero a un nodo. Por ejemplo:

- una declaración genera un nodo `Declaration`
- una asignación genera un nodo `Assignment`
- una operación binaria genera un nodo `BinaryOp`
- un bloque genera un nodo `Block`.

La raíz final es un nodo `Program` que contiene la lista de sentencias del archivo. Esto nos permite preservar la jerarquía del programa fuente y luego imprimir el árbol cuando el análisis termina con éxito.

3. AST y estructura interna

Diseño del AST

La estructura del árbol está implementada en `ast.h`. Para no sobrecomplicar la práctica usé una clase base simple llamada `Node`. Cada nodo almacena:

- el tipo o nombre del nodo, por ejemplo `Program`, `Declaration`, `Assignment` o `BinaryOp`
- un lexema opcional, por ejemplo el nombre de una variable o el operador asociado
- el tipo semántico decorado del nodo
- la lista de hijos.

El árbol no solo representa la forma sintáctica del programa, sino también el resultado del análisis semántico.

Decoración semántica

Para guardar el tipo inferido o validado de cada nodo definimos un enumm:

```
enum class DataType {
    Int,
    Float,
    Invalid
};
```

Cada nodo tiene un campo de este tipo. Por ejemplo:

- un literal entero se crea con tipo `Int`
- un literal float se crea con tipo `Float`
- un identificador toma el tipo asociado en la tabla de símbolos
- una operación binaria calcula su tipo a partir de los operandos
- un nodo con error queda marcado como `Invalid`.

Esto cumple con la parte de la práctica que pide decorar el AST con la información de tipos.

Impresión del árbol

Una vez que el programa termina correctamente, el ejecutable recorre el AST e imprime una representación indentada. Por ejemplo, para una entrada válida se obtiene una salida parecida a:

```
Program
  StatementList
    Declaration [x] <int>
    Declaration [y] <float>
      FloatLiteral [1.2] <float>
    Block
      StatementList
        Declaration [z] <int>
          IntLiteral [1] <int>
```

Con esto sabemos que el parser no solo aceptó el programa, sino que realmente construyó la estructura y que tiene coherencia.

4. Análisis semántico

Tabla de símbolos por ámbitos

La validación semántica se apoya en una tabla de símbolos implementada también en `ast.h`. Internamente tenemos una pila de ámbitos, donde cada ámbito es un diccionario de nombres a símbolos. Cada símbolo guarda al menos:

- el nombre del identificador
- su tipo.

Cuando entra un bloque se hace *push* de un nuevo ámbito y cuando el bloque termina se hace *pop*.

La versión mínima que implementé realiza estos checks:

1. **Redeclaración en el mismo ámbito.** Antes de insertar una variable nueva se consulta únicamente el ámbito actual.
2. **Uso de variable no declarada.** Al leer un identificador se busca desde el ámbito más interno hacia afuera. Si no se encuentra, se reporta error.
3. **Compatibilidad de tipos en asignaciones.** Permitimos `int = int`, `float = float` y `float = int`. En cambio, `int = float` se rechaza.
4. **Operadores aritméticos.** Los operandos de `+`, `-`, `*` y `/` deben ser numéricos.
5. **Condiciones.** En `if` y `while` la condición debe ser numérica.

Promoción de tipos

Para las expresiones aritméticas usamos una regla de promoción muy simple:

- `int op int` produce `int`
- si alguno de los dos operandos es `float`, el resultado es `float`
- si algún operando no es numérico, el resultado queda como `Invalid` y se reporta error.

Motor de errores

En lugar de detener la ejecución en el primer error, almacenamos los errores en un vector global de estructuras `SemanticError`. Cada entrada guarda:

- el mensaje

- la línea
- la columna.

Al terminar el parseo, si el vector no está vacío, el programa imprime todos los errores acumulados y termina con código de salida distinto de cero.

Ejemplos de diagnósticos

Algunos mensajes que ya tenemos son:

```
semantic error: redeclaracion de 'a' en el mismo ambito at line 2, column 6

semantic error: uso de variable no declarada 'a' at line 1, column 1

semantic error: no se puede asignar un valor de tipo float a una variable
    de tipo int at line 2, column 16
```

5. Compilación y pruebas

Construcción del ejecutable

La compilación se hace con un `Makefile`.
El comando de compilación es simplemente:

```
make
```

Pero tienen que estar en la carpeta raíz del proyecto, donde está el `Makefile`.

Forma de ejecución

El programa se usa leyendo desde `stdin`:

```
./validator < examples/ok.morse
```

Casos de prueba

Probamos la implementación con cinco entradas:

Archivo	Intención	Resultado
`ok.morse`	Programa válido con declaraciones, asignaciones y bloque	Acepta e imprime AST
`redecl.morse`	Redeclaración en el mismo ámbito	Error semántico
`type_error.morse`	Asignación de `float` a `int`	Error semántico
`undeclared.morse`	Uso de variable no declarada	Error semántico
`syntax_error.morse`	Falta de punto y coma	Error sintáctico

Salida correcta

En el caso válido, el programa imprime una forma legible del árbol:

```
Program <invalid>
  StatementList <invalid>
    Declaration [x] <int>
    Declaration [y] <float>
      FloatLiteral [1.2] <float>
    Block <invalid>
      StatementList <invalid>
        Declaration [z] <int>
```

```
          IntLiteral [1] <int>
        Assignment [x] <int>
          BinaryOp [+] <int>
            IntLiteral [1] <int>
            IntLiteral [2] <int>
```

0.1. Validación:

- Jose Antonio
  https://youtu.be/b5hcfKk7vg4
- Gerardo Angel
  https://youtu.be/FBtC5lCkmJI